

Desafio Técnico — Time de Processamento em Lote (Batch)

Contexto

Boa parte do nosso trabalho no Batch é pegar dados estruturados vindos de uma fonte e transformá-los em arquivos tabulares prontos para entrega ao cliente. Na prática, isso significa ler um arquivo grande, aplicar regras de negócio e de formatação, e produzir arquivos de saída consistentes — lidando bem com dados que nem sempre vêm limpos.

Este desafio é uma versão reduzida desse problema. A ideia não é te pegar em pegadinha, é ver como você estrutura o código, toma decisões e trata dados do mundo real.

O problema

Você vai receber um arquivo **JSONL** (`sample_clubes.jsonl` , em anexo), onde **cada linha é um objeto JSON** representando um clube de futebol. Cada clube tem um conjunto de dados próprios e uma lista de jogadores.

Seu programa deve ler esse arquivo e gerar **dois arquivos CSV**:

1. `clubs.csv` — um registro por clube (relação 1:1).
2. `players.csv` — um registro por jogador (relação 1:N a partir da lista dentro de cada clube).

Entrada

Cada linha do JSONL tem esta estrutura (exemplo com uma linha "despretensiosa", indentada aqui só para leitura):

```
{
  "club_id": "SCCP",
  "name": "Sport Club Corinthians Paulista",
  "championship": "SERIE A",
  "founding_date": "1910-09-01",
  "city": "São Paulo",
  "state": "SP",
  "country": "Brasil",
  "stadium": "Neo Química Arena",
  "president": "Augusto Melo",
  "nickname": "Timão",
  "colors": ["preto", "branco"],
```

```
"titles": 30,
"players": [
  {
    "player_id": "SCCP-10",
    "name": "Rodrigo Garro",
    "age": 26,
    "goals": 8,
    "debut_date": "2024-01-18",
    "position": "Meia",
    "shirt_number": 10,
    "nationality": "Argentina",
    "market_value": 12000000
  }
]
```

Repare que o JSON traz **mais campos do que os arquivos de saída precisam**. Faz parte do desafio selecionar apenas as colunas pedidas abaixo.

Saída esperada

Atenção: os nomes das colunas nos CSVs são **em português** e diferem das chaves do JSON. Use exatamente os nomes da coluna "Coluna (CSV)" abaixo — incluindo acentos, espaços e maiúsculas/minúsculas —, na ordem em que aparecem.

Arquivo 1 — `clubs.csv` (1:1)

Uma linha por clube, nesta ordem de colunas:

Coluna (CSV)	Origem no JSON	Observação
Id do Clube	club_id	
Nome	name	
Campeonato	championship	
Data de Fundação	founding_date	data no formato yyyy-MM-dd
Cidade	city	
Estado	state	
País	country	
Estádio	stadium	
Presidente	president	
Apelido	nickname	pode estar ausente ou nulo → campo vazio
Cores	colors	lista unida em um só campo (ver regras)

Arquivo 2 — `players.csv` (1:N)

Uma linha por jogador, nesta ordem de colunas:

Coluna (CSV)	Origem no JSON	Observação
Id do Clube	<code>club_id</code> do clube	chave que liga o jogador ao clube
Id do Jogador	<code>players[].player_id</code>	
Nome	<code>players[].name</code>	
Idade	<code>players[].age</code>	
Gols	<code>players[].goals</code>	
Data de Estreia	<code>players[].debut_date</code>	data no formato <code>yyyy-MM-dd</code>
Posição	<code>players[].position</code>	
Número da Camisa	<code>players[].shirt_number</code>	

Regras de negócio e de formatação

- **Filtro por campeonato:** gere dados apenas para clubes que disputam a **Série A** ou a **Série B**. Clubes de qualquer outro campeonato não entram em nenhum dos dois arquivos (nem o clube, nem seus jogadores).
- **Ligação 1:N:** cada linha de `players.csv` carrega o `club_id` do clube a que o jogador pertence. Um clube sem jogadores não gera nenhuma linha em `players.csv`, mas continua aparecendo em `clubs.csv` (se passar no filtro).
- **colors :** a lista de cores deve ser unida em um único campo, separada por `|` (pipe). Ex.: `["preto", "branco"]` → `preto|branco`. Lista vazia ou ausente → campo vazio.
- **Datas:** toda data de saída deve estar em `yyyy-MM-dd`. Se o valor de origem não for uma data válida, deixe o campo **vazio** — a linha continua no arquivo normalmente.
- **Campos vazios:** campos ausentes ou nulos no JSON viram campo vazio no CSV.
- **Formato do CSV:** arquivos em **UTF-8**, com **linha de cabeçalho**, separados por **vírgula**. Campos que contenham vírgula, aspas ou quebra de linha devem ser escapados corretamente (padrão RFC 4180: campo entre aspas duplas, aspas internas duplicadas).
- **Robustez:** o arquivo de exemplo é limpo, mas a base real com que vamos rodar o seu código pode conter **registros malformados ou incompletos**. O programa não deve abortar por causa de um registro problemático: registros inválidos ficam de fora do resultado e o processamento segue para os demais.
- **Volume de dados:** o arquivo de exemplo é pequeno, mas a base real com que vamos rodar o seu código pode ser **muito grande** (muitos milhões de registros). Escreva o programa pensando nesse cenário.

O que entregar

1. **O código** que gera os arquivos.
2. **Os dois CSVs** (`clubs.csv` e `players.csv`) gerados a partir do `sample_clubes.jsonl` em anexo.
3. **Um README** explicando como rodar o programa, recebendo o **caminho do arquivo de entrada por parâmetro** (para facilitar rodarmos com outras bases).

Observações

- Use a **linguagem e as bibliotecas que preferir**. Escolha o que te deixa mais confortável para escrever um código limpo.
- O caminho do arquivo de entrada é um parâmetro do programa; deixe isso documentado no README.
- Você pode usar um assistente de código ou modelo de IA. Se usar, **exporte e envie junto a conversa inteira** utilizada no desenvolvimento. Pedimos isso para avaliar a qualidade e a efetividade do uso da ferramenta: a expectativa é que a IA seja usada como assistente, de forma eficaz — sem despejar o problema inteiro pedindo a solução pronta, e sem se perder em idas e vindas. Queremos ver pedidos diretos e objetivos, que demonstrem seu entendimento do problema, do código e do próprio uso do modelo.

O que valorizamos

- Clareza e organização do código.
- Corretude e consistência dos arquivos gerados.
- Cuidado com dados imperfeitos.
- Eficiência no processamento de arquivos grandes.

Para nos enviar sua resposta, suba o desafio no GitHub em um repositório público e compartilhe o link aqui na caixa de respostas.